BiomationScripter: A Python Library for Developing Synthetic Biology Automation Protocols

Bradley Brown,*† David Markham‡, Jasmine E. Bird,¶ David James Skelton,‡ Chueh Loo Poh,⊥ Anil Wipat,‡ and Irina Dana Ofiteru,*†

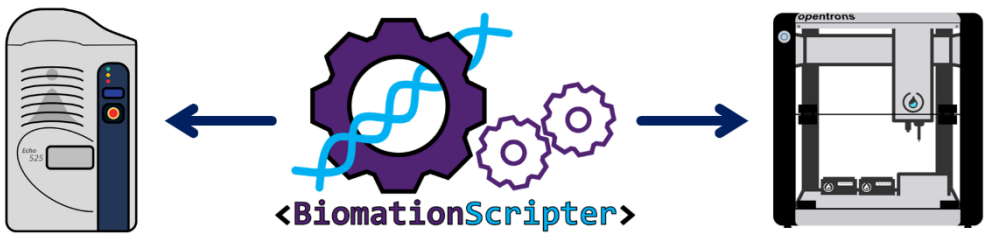
† School of Engineering, Newcastle University, NE1 7RU, United Kingdom

‡ School of Computing, Newcastle University, NE4 5TG, United Kingdom

¶ School of Natural and Environmental Sciences, Newcastle University, NE1 7RU United Kingdom

⊥ Department of Biomedical Engineering and NUS Synthetic Biology for Clinical and Technological Innovation (SynCTI), National University of Singapore, Singapore

*Corresponding Authors: dana.ofiteru@newcastle.ac.uk, bradley.brown4@hotmail.co.uk



Abstract

Synthetic Biology workflow automation has many advantages, such as increased reproducibility of experiments, freeing researchers from repetitive tasks, and making the exploration of large design spaces more feasible. Whilst laboratory automation is becoming more commonplace, the development and re-use of automation workflows and scripts can be difficult. To aid with the development of automated synthetic biology workflows a Python library, BiomationScripter, was developed. BiomationScripter provides tools to allow faster scripting of complex workflows and more efficient and flexible re-use of protocols via BiomationScripter protocol 'Templates'. Whilst initially focusing on only two specific liquid handlers, the BiomationScripter library has been developed with future expansion in mind.

Keywords: Bio-Automation Protocols, Laboratory Automation, Opentrons, Echo 525, Liquid Handling Robot, Synthetic Biology Workflows

Introduction

Automating workflows is crucial for developing the field of synthetic biology[1]–[5]. Despite the popularity of laboratory automation, there are significant barriers to entry[6]. One such barrier is the time required to learn how to program and operate automation equipment, as it is common for equipment to use proprietary software which differ significantly between suppliers[6]–[9]. Furthermore, protocols are often developed in a non-flexible manner preventing efficient re-use of common workflows, such as DNA assembly, even when considering a singular automation platform[10]–[12].

Tools aiding in the development of automation workflows range from open-source platforms providing novel, high-level programming languages allowing biologists to describe an experiment and generate appropriate protocols[12], [13], to proprietary platforms reliant on point-and-click interfaces such as Antha[14]. Whilst these tools have their uses, there remain significant downsides: proprietary efforts tend to focus on specific 'cloud laboratories', preventing usage by researchers with access to their own equipment [10], and a focus on creating simple interfaces for generating automation scripts has often resulted in a lack of flexibility[15].

Presented here is the BiomationScripter Python library v1.0.0 (github.com/intbio-ncl/BiomationScripterLib,)

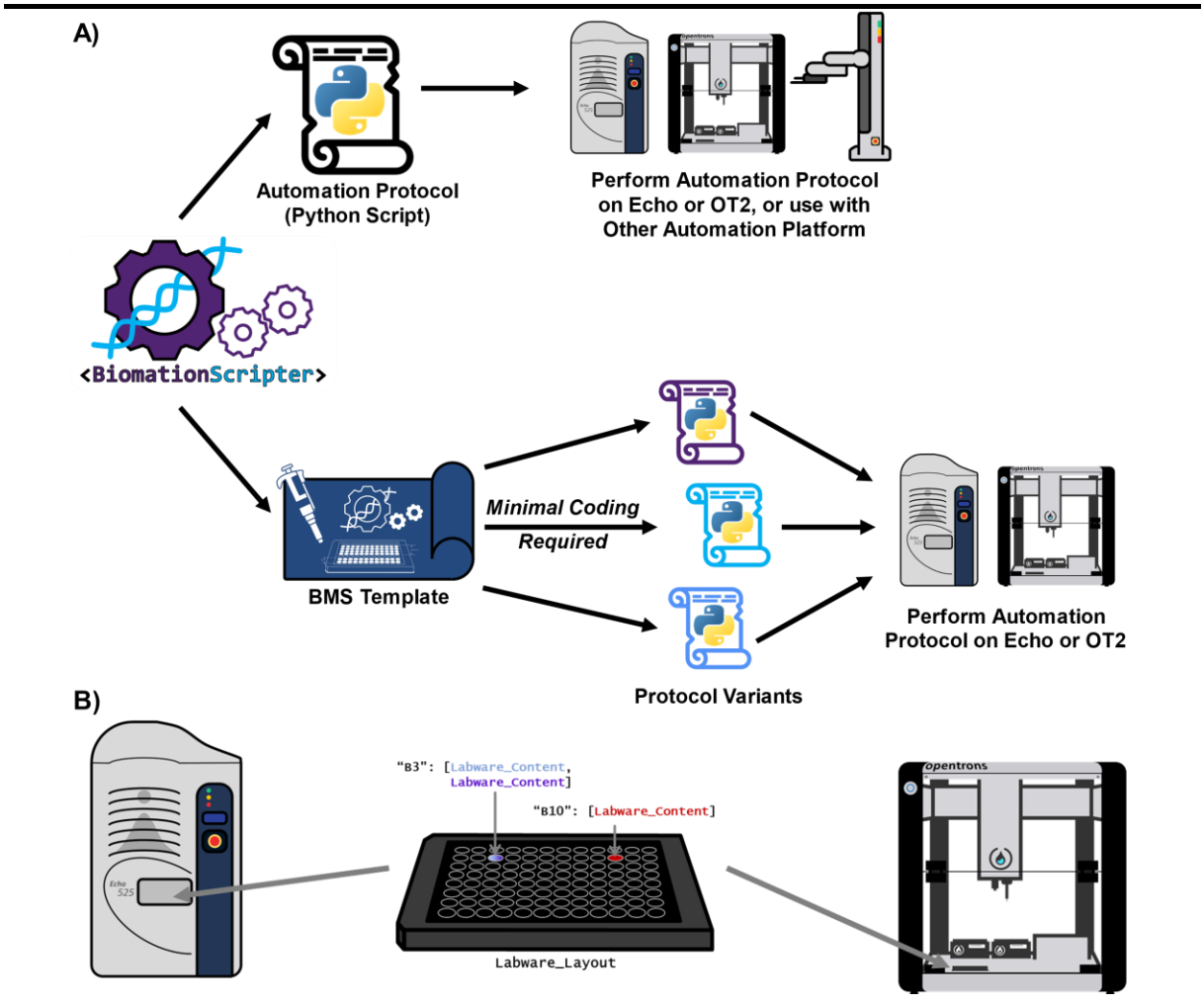


Figure 1. Overview of BiomationScripter Functionality: A) BiomationScripter has two major applications: (i) assisting in the writing of automation protocols for the Echo 525 and Opentrons-2, and in pre-processing information for various other automation platforms, and (ii) providing generalised 'Templates' for common protocols to rapidly generate automation instructions. **B)** Illustration of labware representation in BiomationScripter using the `Labware_Layout` class. A labware's content is specified for each well using the `Labware_Content` class, where each well may have multiple content types. The `Labware_Layout` class can represent labware in different roles, context, and for use in different equipment.

which has two main aims: (i) to provide a set of tools for easier scripting of complex automation protocols, and (ii) to provide a means of developing and sharing automation protocols for easy and flexible re-use (Figure 1A). The first aim is accomplished by the functions and classes within BiomationScripter, and the second by application of BiomationScripter Templates. The library's architecture was designed to allow for further development and collaboration by a wider community, with the aim of providing an automation scripting resource which can grow and adapt over time.

Results and Discussion

The version of BiomationScripter described here focuses on protocol generation for two liquid handlers: Opentrons-2 (OT2; Opentrons) and Echo 525 (Labcyte). Together, the OT2 and Echo 525 cover a wide range of use cases within synthetic biology. The decision was made to develop a programming library for bio-automation as scripting can provide users with a large degree of flexibility when developing automation protocols. The language of choice for BiomationScripter was Python, as this is a relatively lightweight language suitable for scripting of this nature and is already frequently used within synthetic and computational biology [16]. BiomationScripterBiomationScripter is split into three packages: (i) core tools for bio-automation, (ii) specific tools for the OT2 liquid handler (OTProto package), and (iii) specific tools for the Echo 525 liquid handler (EchoProto package). BiomationScripter also supports the development of protocol 'Templates' which are specific to certain types of protocols, such as bacterial heat-shock transformation, which offer fewer flexibility but enable more rapid and standardised development.

BiomationScripter Core Tools

The Core Tools BiomationScripter package contains protocol generation tools for a range of automation equipment. For liquid handling protocols, the presence of labware such as microplates or tube racks is universal. Labware are represented using the 'Labware_Layout' class, storing attributes such as labware type, the number of rows and columns, and the labware's content. Well content is specified using the `Labware_Content` class (Figure 1B). New Labware objects can be generated using Python, or existing labware may be imported via a standard format Excel file (Supplementary File 1).

BiomationScripter also provides a set of helper functions . One example is the `Get_Transfers_Required` function, which uses a `Labware_Layout` object populated as a destination labware to generate a list of liquid transfer actions (Supplementary Figure 1A). Another example is the `Mastermixes_By_Replicates` function, which can be used to flexibly generate mastermixes based on a set of reactions (Supplementary Figure 1B).

OTProto

The OTProto package makes use of the native Opentrons Python library and provides wrapper functions for performing more complex tasks. By providing wrapper functions, OTProto allows users to develop protocols with more complex logic without needing to understand the underlying Opentrons API in great depth or spend time re-coding the required functionality. An example function is 'transfer_liquids' which generates OT2 commands from lists of transfer volumes, source locations, and destination locations. The function automatically selects which pipette to use for each transfer. Functions in OTProto contain optional arguments allowing for flexible use.

A full set of OTProto functions and classes can be found in the documentation.

EchoProto

The Echo 525 is programmed using proprietary software packages. However, a .csv file termed a 'Picklist' can be uploaded containing all the transfer steps. The BiomationScripter EchoProto package provides a Python interface for creation of Echo 525 Picklists. It requires a set of source plates, which specify the type, quantity, and location of all source materials, and a set of destination plates, which specify the desired final state once all liquid has been transferred. EchoProto can handle aliquots of the same source material and will use volume tracking to move onto the next well after the previous is depleted. In the case of a lack of source material, the user will be prompted and informed which reagents are lacking and told how much extra is required. The user will also be alerted to any wells which are below the working range for that plate.

Templates

BiomationScripter Templates are generalised protocols of specific applications for repeated use with slight variations between runs. For example, preparation of PCR reactions generally follows the same steps, however the specific DNA, primers, polymerase, and buffers may vary, along with other user-defined parameters such as the number of reactions prepared, repeats, and final volume. Each Template has required inputs and a variety of optional inputs allowing advanced users more flexibility. Examples for an EchoProto Template (Loop Assembly) and an OTProto Template (Standard Plate Reader Calibration) are shown in Figure 2. New Templates can be created by users with an intermediate knowledge of Python and added to the library by following walkthroughs in the documentation.

Methods

BiomationScripter is a Python library developed and tested using Python 3.9.7 and can be installed following the instructions in the documentation (see Supplementary Information). The EchoProto and OTProto packages each have a further sub-package within which Template protocol files are defined. The EchoProto package generates .csv files which can be imported in the Echo Plate Reformat software as Picklists. The architecture used by EchoProto is shown in Supplementary Figure 2. The OTProto package uses the Opentrons Python API

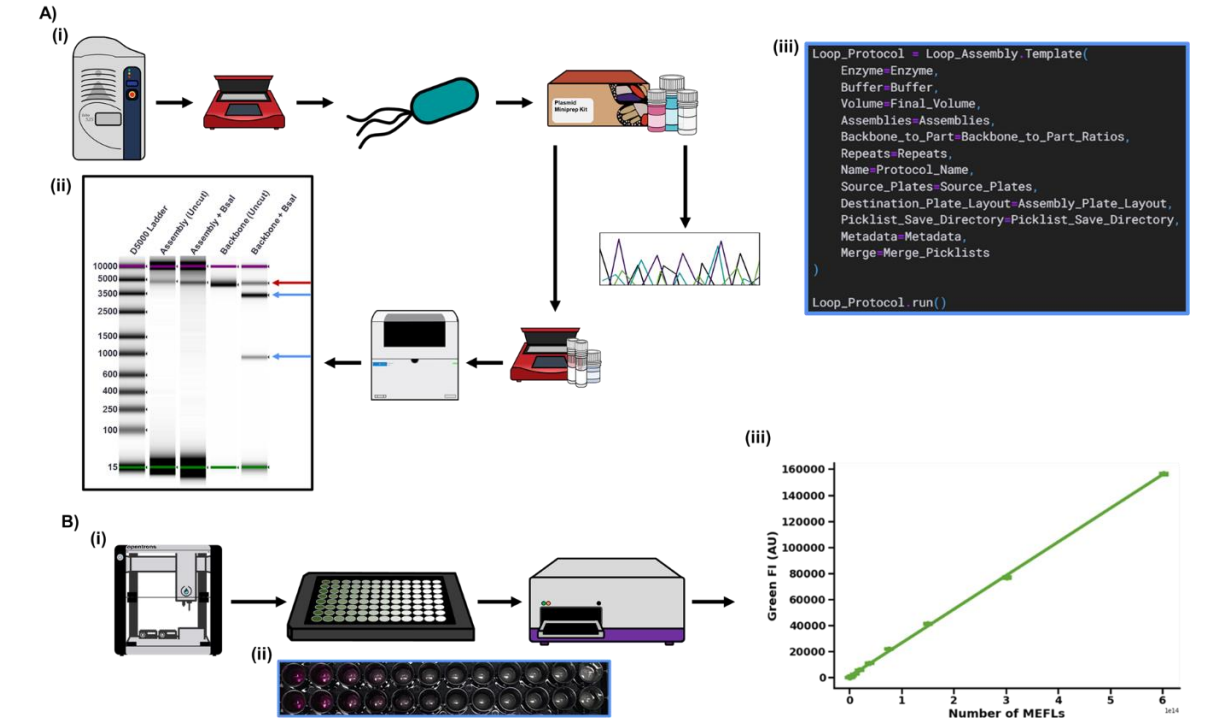


Figure 2. BiomationScripter Template Examples: Examples of how Templates can be used to automate common protocols. Full methods are described in Supplementary File 2. **A)** Automated Golden Gate DNA assembly using the Echo 525 and Loop Assembly Template. (i) Workflow overview: DNA assemblies performed on the Echo 525 using BiomationScripter and a 96-well plate compatible thermocycler. DNA assemblies were verified by bacterial transformation, plasmid purification, and a combination of Sanger sequencing and restriction digest confirmation. (ii) DNA gel obtained by the Tapestation from a confirmation restriction digest of DNA assemblies. Red arrow shows sizing for the assembly backbone, light blue arrows show sizing for the linear fragments after digestion of DNA assemblies. (iii) Code fragment from the Loop (Golden Gate-based) DNA assembly Template, showing the required user inputs. **B)** Automated preparation of a 96-well plate for standard calibration of multi-modal plate readers [17] using the OT2 and iGEM Standard Calibration BiomationScripter Template. (i) workflow overview: OT2 used to prepare calibrants in a 96-well plate, which was measured using a plate reader. (ii) Output of a 96-well plate prepared by an OT2 using the BiomationScripter Template (dyed water used for visualisation). (iii) Standard curve output for calibrating green fluorescence measurements.

(version 2.11) to directly control the OT2 at runtime. Therefore, the BiomationScripter library must be installed on the OT2, following instructions in the documentation (see Supplementary Information). For both OTProto and EchoProto, protocol Templates are developed as classes by extending a Template superclass.

The full methods used to illustrate utility of the EchoProto and OTProto Templates shown in Figure 2 can be found in Supplementary File 2.

Supplementary Information

The BiomationScripter documentation is available at https://biomationscripterlib.readthedocs.io/en/latest/.

Supplementary File 1: Standard_Labware_File.xlsx
Supplementary File 2: Methods.docx
Supplementary Figures: Supplementary Figures.docx

Author Contributions

B.B. provided the initial concept for BiomationScripter, with all authors contributing towards discussions and providing valuable insights. B.B. and D.M. wrote the BiomationScripter code, with J.E.B. and D.J.S. providing contributions. B.B., D.M., and J.E.B. performed testing of BiomationScripter for generating automated workflows. B.B. and D.M. performed experimental testing of protocols generated using BiomationScripter. B.B. wrote the documentation and walkthrough tutorials with assistance from D.M. and J.E.B. All the authors contributed to the writing and editing of this manuscript and created all figures in their entirety.

Funding

B.B. was funded by an EPSRC DTP fellowship.

References

[1] D. Bryce *et al.*, "Round Trip: An Automated Pipeline for Experimental Design, Execution, and Analysis," *ACS Synth Biol*, vol. 11, no. 2, pp. 608–622, Feb. 2022, doi: 10.1021/ACSSYNBIO.1C00305/SUPPL_FILE/SB1C00305_SI_001.PDF.

[2] J. A. Yeow, P. K. Ng, K. S. Tan, T. S. Chin, and W. Y. Lim, "Effects of Stress, Repetition, Fatigue and Work Environment on Human Error in Manufacturing Industries," *Journal of Applied Sciences*, vol. 14, no. 24, pp. 3464–3471, Dec. 2014, doi: 10.3923/JAS.2014.3464.3471.

[3] J. Zhang *et al.*, "Accelerating strain engineering in biofuel research via build and test automation of synthetic biology," *Curr Opin Biotechnol*, vol. 67, pp. 88–98, Feb. 2021, doi: 10.1016/J.COPBIO.2021.01.010.

[4] M. M. Jessop-Fabre and N. Sonnenschein, "Improving reproducibility in synthetic biology," *Front Bioeng Biotechnol*, vol. 7, no. FEB, p. 436850, Feb. 2019, doi: 10.3389/FBIOE.2019.00018/BIBTEX.

[5] P. Carbonell, T. Radivojevic, and H. García Martín, "Opportunities at the Intersection of Synthetic Biology, Machine Learning, and Automation," *ACS Synth Biol*, vol. 8, no. 7, pp. 1474–1477, Jul. 2019, doi: 10.1021/ACSSYNBIO.8B00540/ASSET/IMAGES/LARGE/SB-2018-00540F_0002.JPEG.

[6] N. Hillson *et al.*, "Building a global alliance of biofoundries," *Nature Communications 2019 10:1*, vol. 10, no. 1, pp. 1–4, May 2019, doi: 10.1038/s41467-019-10079-2.

[7] S. M. Brooks and H. S. Alper, "Applications, challenges, and needs for employing synthetic biology beyond the lab," *Nature Communications 2021 12:1*, vol. 12, no. 1, pp. 1–16, Mar. 2021, doi: 10.1038/s41467-021-21740-0.

[8] D. Kumar, K. Achuthan, B. Nair, and S. Diwakar, "Online bio-robotics labs: Open hardware models and architecture," *International Conference on Robotics and Automation for Humanitarian Applications, RAHA 2016 - Conference Proceedings*, May 2017, doi: 10.1109/RAHA.2016.7931877.

[9] M. A. Torres-Acosta, G. J. Lye, and D. Dikicioglu, “Automated liquid-handling operations for robust, resilient, and efficient bio-based laboratory practices,” *Biochem Eng J*, vol. 188, p. 108713, Dec. 2022, doi: 10.1016/J.BEJ.2022.108713.

[10] E. Appleton, D. Densmore, C. Madsen, and N. Roehner, “Needs and opportunities in bio-design automation: four areas for focus,” *Curr Opin Chem Biol*, vol. 40, pp. 111–118, Oct. 2017, doi: 10.1016/J.CBPA.2017.08.005.

[11] L. Gautier, “Reproducibility in the lab: the proof is in the protocol,” *Biotechniques*, vol. 73, no. 2, pp. 71–74, Aug. 2022, doi: 10.2144/BTN-2022-0074/ASSET/IMAGES/LARGE/FIGURE2.JPEG.

[12] G. Linshiz, N. Stawski, S. Poust, C. Bi, J. D. Keasling, and N. J. Hillson, “PaR-PaR laboratory automation platform,” *ACS Synth Biol*, vol. 2, no. 5, pp. 216–222, May 2013, doi: 10.1021/SB300075T/SUPPL_FILE/SB300075T_SI_002.ZIP.

[13] V. Ananthanarayanan and W. Thies, “Biocoder: A programming language for standardizing and automating biology protocols,” *Journal of Biological Engineering 2010 4:1*, vol. 4, no. 1, pp. 1–13, Nov. 2010, doi: 10.1186/1754-1611-4-13.

[14] “Antha: a platform for engineering biology - The Official PLOS Blog.” https://theplosblog.plos.org/2017/02/antha-a-platform-for-engineering-biology/ (accessed Sep. 24, 2023).

[15] “GitHub - PyLabRobot/pylabrobot: A hardware agnostic interface and developer ecosystem for lab automation.” https://github.com/pylabrobot/pylabrobot (accessed Sep. 24, 2023).

[16] J. W. Yeoh *et al.*, “SynBiopython: an open-source software library for Synthetic Biology,” *Synth Biol*, vol. 6, no. 1, Feb. 2021, doi: 10.1093/SYNBIO/YSAB001.

[17] J. Beal *et al.*, “Multicolor plate reader fluorescence calibration,” *Synth Biol*, vol. 7, no. 1, pp. 1–9, Dec. 2022, doi: 10.1093/SYNBIO/YSAC010.